

Relatório do chat — Hermes Creative Workstation

Data de consolidação: 9 de outubro de 2026, horário de São Paulo.

Este relatório reúne o histórico disponível deste chat e os registros persistidos no projeto. É uma consolidação técnica, não uma transcrição literal de todas as mensagens ou comandos. Distingue implementação, execução real, testes focados e qualificação de produto. Os documentos de 8 de outubro representam checkpoints históricos: suas declarações de bloqueio ou de ausência de implementação não descrevem necessariamente os avanços posteriores.

1. Pedido e objetivo

O pedido inicial foi executar integralmente o texto “PROMPT MESTRE — IMPLEMENTAÇÃO INTEGRAL DO HERMES CREATIVE WORKSTATION”, anexado em:

```
C:\Users\Kevyn Lucas\.codex\attachments\4869177f-2348-48b4-b63f-a34b56730f9e\Texto colado.txt .
```

O objetivo evoluiu para avançar da CW-02 até o final, preservando a arquitetura existente do Hermes. O escopo inclui runtime de engines, projetos criativos, imagens, vídeo, Remotion opcional, Penpot, Three.js, engines adicionais e aprendizado/reuso pelo Experience Compiler.

As regras determinantes foram: upstream-first, alterações separadas e revisáveis, reutilização dos owners canônicos, preservação de autoridade por perfil/sessão/tarefa, evidência independente dos outputs e ausência de merge automático. A autorização posterior permitiu desenvolvimento com baseline pendente, sem transformar falhas em aprovação.

2. Sequência das decisões do usuário

1. Solicitou seguir o texto colado.
2. Esclareceu que o uso atual é pessoal; pode haver uso interno empresarial no futuro e uma chance baixa de distribuição comercial.
3. Perguntou por que a CW-01 estava bloqueada.
4. Perguntou quais problemas realmente impediam a implementação.
5. Perguntou se poderia autorizar registrar as pendências como “Não resolvido” e seguir para implementação.
6. Respondeu “Está autorizado”. Foi registrada uma exceção explícita de desenvolvimento.
7. Pediu um balanço do que já havia sido feito. Foi informado que havia código e provas parciais, mas o produto completo não estava pronto.
8. Solicitou este relatório completo.

A declaração de uso pessoal esclareceu o cenário atual de licenciamento do Remotion. Não aprovou automaticamente o uso futuro empresarial, redistribuição, instalação de engines ou qualquer default inseguro de execução.

3. Por que a CW-01 foi bloqueada

A CW-01 foi uma auditoria de baseline, owners, upstream, dependências e licenças. O bloqueio inicial decorreu das regras do repositório, que exigem uma base upstream-aligned qualificada antes de expansão funcional.

Foram encontrados:

- Drift material de upstream em áreas de processo, MCP, resolver e Electron.
- Ausência de baseline composto qualificado no SHA exato.
- CI incompleto ou vermelho em checks aplicáveis.
- Auditoria de dependências de produção com 18 achados naquele checkpoint: 2 críticos, 4 altos, 3 moderados e 9 baixos. Essa é uma observação histórica, não uma auditoria refeita na data deste relatório.
- Incidente KI-024 de autoridade de entrada/voz e pendência KI-025 de startup, sem fechamento comprovado neste trabalho.
- H-080B.3 e prova nativa/empacotada ainda incompletos.
- Problemas de ambiente Python e proveniência de Laya.

Havia distinções importantes: a falha do PR de documentação #50 ocorreu na auditoria de dependências de produção, não foi demonstrada como erro do compilador TypeScript; checks de outra branch ou outro SHA não qualificavam a base Creative. O mesmo identificador KI-024 tinha significados diferentes em registros de branches distintas e não podia ser transferido entre elas.

A auditoria também identificou infraestrutura reutilizável: AppResolver, ProcessRegistry, WorkerRegistry, BrowserTask/WebContentsView, ScopedPolicyEngine, TaskCompiler, ArtifactStore, ExecutionJournal e Experience Compiler. A proposta foi estender esses owners, sem criar executor, banco de autoridade ou editor gráfico paralelo.

4. Stage A e exceção de desenvolvimento

O trabalho de baseline foi mantido separado da implementação. Houve composição de reparos identificados nos registros como R1/R2, inspeção de CI e diagnóstico de merge. O merge virtual apresentou inicialmente 55 conflitos e, após a composição, 56, incluindo o lockfile. Isso foi diagnóstico por merge-tree, não integração concluída.

O pin selecionado permaneceu imutável:

517b5e10febd619ce30bb22580e29b160266eb43 .

Ele não foi declarado adotado ou qualificado. A última observação registrada de upstream foi:

1e0c7730d791f5ce855c5c78935cfea6fb1e43a9 .

O drift posterior foi inspecionado e classificado para ciclo seguinte, sem iniciar um segundo sync. Entre os deltas estavam alterações amplas de lint e correções de owner de home em definições de serviço do Gateway relevantes para futura qualificação de perfis.

Sua autorização foi registrada em `workstation/creative-workstation/DEVELOPMENT_EXCEPTION_2026-10-08.md` . Seu alcance foi:

- Permitir a implementação na lane de desenvolvimento, começando pela CW-02.
- Manter upstream, conflitos, CI, ambiente, KI-024/KI-025 e H-080B.3 como não resolvidos.
- Não autorizar merge na main, promoção de capability, dispensa de isolamento, licença ou verificação.

Consequentemente, a CW-01 continua não qualificada, mas deixou de impedir todo o desenvolvimento autorizado. O status histórico “BLOCKED” deve ser lido junto dessa exceção posterior.

5. CW-02 — runtime de engines

Foi implementada uma base opt-in para descobrir e executar engines externas:

- `creative_runtime.py` : descoberta passiva por infraestrutura canônica de resolução.
- `creative_apps.py` : manifesto tipado/versionado, executável absoluto, fingerprint SHA-256 e probes fixos por engine.
- `creative_process.py` : execução sem shell, ambiente por allowlist, validação de escopo e reaproveitamento do ProcessRegistry.
- Flag `creative.enabled` , desligada por padrão, subordinada ao Workstation habilitado.
- Workspace privado do perfil; rejeição de paths fora do escopo, identidade incorreta e drift de executável.
- Lifecycle, logs, checkpoint, recuperação e cancelamento de árvore de processos pelo owner existente.

A descoberta não instala nem inicia engines. Não foi criado um novo core tool com argv arbitrário. Scripts não confiáveis não receberam autorização genérica por essa API.

Evidência registrada: 6 testes focados passaram, incluindo Node real, health inválido apesar de exit zero, opt-out, autoridade negada antes de spawn, troca de perfil $A \rightarrow B \rightarrow A$, exclusão de variáveis sensíveis, timeout, cancelamento e recuperação.

Limite: probes CLI não qualificam serviços de rede, portas, UI ou o produto completo.

6. CW-03A — projetos, revisões e imagens nativas

Foi implementado um formato declarativo restrito para convite com canvas, retângulos, círculos e texto. Há validação de geometria, tamanho, cores, fontes e campos desconhecidos; texto é escapado.

Os projetos possuem revisões imutáveis, source JSON e manifesto. A gravação é exclusiva, com hashes e manifesto como último marcador de commit. Alterações humanas detectadas por hash são recusadas em vez de sobrescritas silenciosamente.

A execução valida TaskRun real, claim, perfil, sessão e workspace; registra preparação no ExecutionJournal e publica outputs no ArtifactStore. Após efeitos parciais ou despacho incerto, retorna efeito incerto com operation ID, sem retry automático.

A renderização usa BrowserTask e o mesmo WebContentsView nativo do Hermes, com vínculo explícito à tarefa, controles de autoridade e respeito ao controle humano. O readback consulta o estado canônico em `Runtime/browser-session.json`. A verificação independente com Pillow confere PNG, dimensões, hash e decodificação; isso não equivale a certificação automática VERIFIED.

Foi criada uma CLI em `python -m workstation.creative` para salvar, inspecionar, renderizar e posteriormente gerar vídeo. Ela depende do contexto herdado legítimo de sessão/tarefa; os argumentos não criam autoridade.

Houve evidências PNG/SVG e testes focados de projeto, mídia e broker. Entretanto, a captura nativa apresentou instabilidade durante a sequência de criação, reabertura e variação, gerando uma rodada adicional de correção descrita abaixo.

7. CW-03B — vídeo real com FFmpeg

Foi implementada exportação tipada de PNG para vídeo estático H.264/MP4. Não se trata ainda de animação Remotion.

O adapter reaproveita política, aprovação, TaskRun, ProcessRegistry, ArtifactStore e journal. Revalida autoridade durante a espera e encerra o filho próprio quando a autoridade muda, há timeout ou interrupção.

Há limites de duração, fps, frames, dimensões, output e tempo, argumentos fixos e protocolos file/pipe. Esses limites não são uma quota total de RAM imposta pelo sistema operacional.

Foi utilizado FFmpeg/ffprobe já instalado, versão 8.1.1 full Gyan. Os hashes foram calculados nos executáveis reais, não nos shims Chocolatey. A inspeção identificou GPLv3 e libx264, sem

`enable-nonfree`. Não houve redistribuição de binário nem aprovação genérica para distribuição futura.

Resultado real registrado:

Propriedade	Resultado
Container / codec	MP4 / H.264
Pixel format	yuv420p
Dimensões	360 × 640
Duração	8 segundos
Frame rate	30 fps
Frames decodificados	240
Tamanho	15.242 bytes
Erro médio RGB do preview	aproximadamente 2,0294/255

O `ffprobe` verificou stream, container, timing, dimensões, tamanho e frames. O primeiro frame foi decodificado e comparado ao PNG, além de inspeção visual. Também houve execução pela CLI real com configuração de perfil.

Uma falha foi encontrada: `ProcessRegistry.wait` retorna cauda de log, que podia eliminar o cabeçalho de versão. Foi corrigido usando leitura completa e limitada do log do owner. Um teste preserva o cabeçalho mesmo após mais de 5.000 caracteres; outro altera a autoridade da tarefa com subprocesso vivo e comprova encerramento.

Limitação relevante: a evidência de vídeo usou PNG histórico copiado para o workspace. Ainda não foi implementada a passagem direta por referência do `ArtifactStore` do PNG nativo recém-renderizado para o vídeo na mesma tarefa. Esse próximo passo foi planejado, não executado.

8. CW-03C — Remotion: fontes prontas, execução pendente

Foi criada a base de fontes editáveis, sem instalar ou executar o Remotion:

- Schema restrito de convite, incluindo texto, paleta, dimensões, duração, fps e geometria.
- Template TSX original com entrada animada e pulso do logo.
- Identidade de engine no projeto, sem troca silenciosa na mesma linhagem.
- Persistência de `Invitation.tsx`, `index.tsx` e `package.json`, com hashes individuais.
- Publicação das fontes pelo `ArtifactStore/journal`.
- `save --engine remotion` e inspeção da engine.
- Recusa do adapter SVG quando recebe projeto Remotion.

- Tratamento de escrita parcial como efeito incerto, sem publicar sucesso.

Foram registrados 9 contratos aprovados com filesystem, DB e ArtifactStore reais, incluindo injeção de falha após escrita parcial. A sintaxe TSX foi empacotada com esbuild e dependências externas; isso não é typecheck do SDK instalado nem prova de render Remotion.

A versão candidata é Remotion 4.0.534, com React/ReactDOM 19.2.7. Foi baixado apenas o arquivo publicado do renderer para inspeção estática, com scripts desabilitados e integridade conferida. Não houve importação ou execução desse pacote.

Foram identificados dois pontos concretos de isolamento nos defaults do renderer: bind em interfaces wildcard e flags que desabilitam sandbox do navegador. Eles não foram aceitos como fronteira final de execução. Falta adapter com isolamento apropriado, acesso local/owner e lifecycle limitado, seguido de provas reais positivas e negativas.

Foi enviada uma pergunta específica para autorizar instalação local isolada do Remotion e dependências, sem instalação global nem inclusão no produto distribuído. Não há resposta registrada no histórico disponível. A autorização de desenvolvimento anterior não foi tratada como consentimento específico para essa instalação.

9. Correção adicional da captura nativa

A falha original `creative_paint_timeout` foi reproduzida. A implementação anterior aguardava fontes e frames de pintura, depois usava `capturePage` numa view estacionada com apenas uma pequena região exposta.

As tentativas tiveram os seguintes resultados:

1. Captura CDP na mesma view permitiu alguns passos, mas a variação ainda apresentou timeout.
2. Alterar `captureBeyondViewport` isoladamente não resolveu toda a sequência.
3. Usar `fromSurface: false` produziu imagem branca. O oráculo de pixels recusou corretamente a saída.
4. Foi adotada geometria temporária de captura na janela existente, com a mesma view no fundo da pilha, sem criar janela/browser paralelo.
5. A geometria, zoom e ordem são restaurados; views reparentadas não são puxadas de volta. Um lock transitório por host impede capturas simultâneas de corromper a restauração.
6. O estacionamento final passou a considerar a geometria atual da janela, porque ela pode mudar durante a execução.

O teste nativo mais recente passou por criação, reabertura em outro processo e variação. Verificou pixels esperados, estabilidade do hash após reabertura, diferença da variação, preservação de foco e foreground humano, ordem de views, zoom, geometria e controles negativos de sessão/autoridade.

Foram registrados 47 testes aprovados em 7 arquivos do Desktop. Há também contratos de revogação de owner, timeout e captura tardia sem publicação, concorrência e reparenting.

Essa correção está local e ainda não foi commitada/publicada em PR. Faltam a consolidação de evidência durável, as verificações finais após a última edição e qualificação aplicável. Um typecheck anterior passou, mas não deve ser apresentado como prova do estado final depois de todas as alterações.

10. Falhas de ambiente e revisão automática

- A suíte ampla do Workstation falhou com 7 falhas de proveniência de Laya em 4 arquivos no Python compartilhado. Não foram escondidas nem tratadas como aprovação.
- A preparação do ambiente Python local da worktree sofreu DNS/download de Torch e uma tentativa longa foi cancelada. Não há ambiente locked completamente qualificado.
- Uma invocação Vitest com nome de projeto inexistente falhou antes da coleta; foi corrigida para o script canônico do Desktop.
- Houve impedimento temporário por quota em uma tentativa de publicação, seguido de sucesso após liberação.
- Uma revisão automática de execução expirou; nova tentativa foi aceita.
- A revisão automática rejeitou uma proposta que removia verificações repetidas de owner ao redor da captura CDP. A alteração não foi aplicada. A solução posterior manteve as verificações antes e depois da captura, além dos controles de linhagem e controle humano, e foi aceita.

A rejeição não permanece como bloqueio de toda a tarefa: uma alternativa segura foi implementada. Nenhum desses episódios autoriza enfraquecer os guardrails.

11. Branches, PRs e preservação

PR	Finalidade registrada	Situação de entrega
#50	Fundação documental consultada	Referência histórica; não incorporada wholesale
#57	Stage A / reparos e preflight	Draft; baseline não qualificado
#58	CW-02 runtime	Draft publicado
#59	CW-03A projetos e render	Draft publicado
#60	CW-03B vídeo	Draft publicado

PR	Finalidade registrada	Situação de entrega
#61	CW-03C fontes Remotion	Draft publicado

Os estados acima refletem os últimos registros deste trabalho; não foi feita consulta nova ao GitHub para afirmar o estado atual de todos os checks. CI parcial de um PR não foi emprestado para outro SHA.

A implementação está na worktree separada:

```
C:\Users\Kevyn Lucas\.codex\worktrees\creative-stage-a\hermes-agent .
```

Na conferência feita para este relatório, a branch ativa era `codex/creative-native-capture-20261008`, com alterações em runtime do Browser, frame criativo, fixture nativa, journal e CLI, além de dois novos arquivos de testes. Isso confirma que a última correção continua sem commit.

Os arquivos pessoais do checkout original, incluindo workspace Obsidian e nota de dogfood, foram preservados. Não houve merge automático na main nem descarte destrutivo de trabalho concorrente.

12. Evidências e documentos existentes

Na worktree, `workstation/creative-workstation/` contém auditoria CW-01, exceção de desenvolvimento, runtime CW-02, documentos CW-03A/B/C, plano, arquitetura, integrações, matriz de verificação e pesquisa original.

Evidências duráveis estão em:

- `evidence/cw01-20261008/` : auditorias, conflitos e recibos de CI históricos.
- `evidence/cw03a-20261008/` : convite PNG/SVG, variação, fonte e readbacks históricos.
- `evidence/cw03b-20261008-trial1/` : MP4, preview e receipt.
- `evidence/cw03b-20261008-cli/` : MP4 e receipt pelo endpoint real da CLI.

O último teste nativo da correção gerou estado em pasta temporária `hermes-creative-project-native-74QtQU`. A cópia de sua evidência para localização durável ainda estava pendente. Os paths de perfil temporário dentro de receipts históricos não devem ser confundidos com arquivos ainda existentes: os outputs portáteis copiados são a evidência preservada.

13. Estado por fase ao encerrar este relatório

Fase	O que existe	O que falta
CW-00	Fundação documental e análise	Não substitui implementação
CW-01	Auditoria e diagnóstico de baseline	Adoção do pin, conflitos, ambiente e qualificação completa
CW-02	Runtime opt-in implementado e testes focados	Qualificação integrada/produto; contratos de serviços quando aplicáveis
CW-03A	Projetos, PNG/SVG e prova nativa recente	Publicar correção final e fechar gates aplicáveis
CW-03B	MP4 real validado e CLI	Handoff direto de artefato nativo, falhas/recovery e qualificação final
CW-03C	Fontes e revisões Remotion	Consentimento de instalação, auditoria completa, isolamento e render real
CW-04	Pesquisa inicial Penpot	Conexão autorizada, adapter e E2E de edição humano/agente
CW-05	Pesquisa inicial Three.js	Bridge tipada, cena, revisão/undo, GLB e E2E
CW-06	Descoberta de engines prevista no runtime	Adapters operacionais e provas de Inkscape/Blender; demais opcionais
CW-07	Owners existentes identificados	Vertical qualificada, replay, promoção e reuso criativo real

Não foram medidos custos LLM ou ganhos de reuso suficientes para declarar redução de custo por resultado verificado. Não há certificação criativa promovida pelo Experience Compiler neste histórico.

14. Próximos passos concretos

1. Consolidar evidência e verificações finais da correção de captura, commitá-la e publicar PR separado.
2. Conectar PNG do ArtifactStore ao vídeo com referência tipada e validação da mesma tarefa, evitando cópia manual e acesso arbitrário fora do workspace.
3. Resolver consentimento e isolamento do Remotion antes de instalação/execução; executar render e verificação independente quando admitidos.
4. Implementar Penpot, Three.js e engines adicionais em unidades revisáveis, com revisão humana preservada e provas reais.
5. Integrar o fluxo qualificado ao Experience Compiler existente, sem auto-certificação.
6. Fechar baseline e gates de promoção antes de qualquer declaração de produto pronto ou merge.

15. Conclusão factual

O chat produziu código, PRs, imagens, vídeo real, testes de autoridade e recuperação, além de diagnóstico de falhas e uma correção nativa recente. A entrega ultrapassou planejamento e documentação.

O Hermes Creative Workstation integral ainda não foi concluído, qualificado ou integrado na main. A autorização permitiu avançar com pendências visíveis; não eliminou essas pendências. O estado correto é desenvolvimento parcial com provas concretas em algumas verticais e gates de produto ainda abertos.

Prompt usado

PROMPT MESTRE — IMPLEMENTAÇÃO INTEGRAL DO HERMES CREATIVE WORKSTATION

0. MISSÃO E AUTORIZAÇÃO DE EXECUÇÃO

Você é o agente engenheiro responsável pela implementação integral do **Hermes Creative Workstation**, uma extensão do Hermes Workstation existente.

Repositório:

`kevynlucasprofissional-stack/hermes-agent`

Diretório da iniciativa:

`workstation/creative-workstation/`

Plano inicial: PR #50

<https://github.com/kevynlucasprofissional-stack/hermes-agent/pull/50>

Branch de documentação:

`docs/creative-workstation-foundation-20261008`

Objetivo

Transformar o Hermes Work em uma estação criativa integrada na qual o agente possa criar, modificar, visualizar, renderizar, verificar, preservar e reutilizar projetos produzidos por diferentes engines especializadas.

O usuário e o agente deverão poder trabalhar nos mesmos projetos editáveis, preservando revisões, arquivos nativos, assets, operações e evidências.

O Hermes deverá aproveitar sua arquitetura operacional existente, incluindo Control Plane, TaskCompiler, OperationalCapabilityRegistry, Experience Compiler, ArtifactStore, ExecutionJournal, Chromium/Electron, plugins, skills e MCP.

Ordem de execução obrigatória

EXECUTE A INICIATIVA COMPLETA, COMEÇANDO PELA CW-01 E AVANÇANDO ATÉ A CW-07.

A CW-01 não é a tarefa final. É a primeira fase de uma execução sequencial.

Ordem:

1. CW-01 — Auditoria, baseline, owners, segurança e liberação.
2. CW-02 — Creative Runtime, descoberta, lifecycle e health.
3. CW-03A — React/SVG, Chromium real, PNG e persistência.
4. CW-03B — FFmpeg/ffprobe e processamento audiovisual.
5. CW-03C — Remotion, condicionado à aprovação de licenciamento e segurança.
6. CW-04 — Penpot, MCP e edição colaborativa.
7. CW-05 — Three.js Editor, bridge tipada e edição de cenas 3D.
8. CW-06 — Engines especializadas, Creative Project Manifest e consolidação.
9. CW-07 — Integração criativa ao Experience Compiler e reutilização operacional.

As subdivisões CW-03A/B/C fazem parte da CW-03.

Não pare voluntariamente após CW-01, CW-02 ou qualquer outra fase.

Ao concluir e qualificar uma fase, registre seu resultado, crie o PR correspondente quando aplicável e avance para a próxima fase elegível, sem exigir que o usuário envie um novo prompt.

Não confunda autonomia de execução com autorização irrestrita. Nenhuma instrução deste prompt substitui os gates canônicos, as permissões exigidas ou as decisões humanas reservadas.

Se encontrar um bloqueio real:

- Identifique sua causa e seu impacto exato.
- Execute as correções que estejam dentro da autoridade e do escopo permitido.
- Reexecute os testes necessários.
- Continue se o bloqueio tiver sido comprovadamente resolvido.
- Se não puder resolvê-lo com segurança, interrompa apenas o trabalho dependente desse bloqueio.

- Prossiga com atividades independentes e permitidas.
- Não declare etapas bloqueadas como concluídas.
- Não use pendências cosméticas ou questões já resolvidas pela documentação como pretexto para interromper toda a execução.

Não faça merge automaticamente no `main`.

1. PRINCÍPIOS DE EXECUÇÃO

Esta iniciativa já possui visão, prioridades e arquitetura documentadas.

Portanto:

Não reinicie a discussão arquitetural. Não substitua o plano existente por uma proposta nova. Implemente a solução documentada.

Adaptações só são permitidas quando a inspeção do código atual, os contratos existentes, as limitações comprovadas ou os testes demonstrarem que são necessárias.

Adote os seguintes princípios:

- **Upstream-first:** preservar compatibilidade com o Hermes Agent e seguir os procedimentos de sincronização estabelecidos.
- **First-party seams:** integrar tecnologias externas pelos pontos de extensão legítimos do Hermes.
- **Menor mudança suficiente:** evitar refatorações e abstrações desnecessárias.
- **Sem autoridades paralelas:** não duplicar registries, approvals, journals, sessões ou mecanismos de execução.
- **Projetos editáveis:** preservar os fontes e não apenas outputs renderizados.
- **Evidência empírica:** demonstrar funcionamento real, não apenas compilação ou testes simulados.
- **Segurança por padrão:** autorização explícita para ações privilegiadas e efeitos externos.
- **Reutilização operacional:** aproveitar capacidades existentes antes de introduzir novas.
- **Mudanças reversíveis:** suporte a rollback, cancelamento, retomada e diagnóstico.
- **Documentação sincronizada:** registrar decisões e estado efetivo conforme a implementação progride.

Não faça mudanças amplas por conveniência. Não introduza dependências apenas porque elas poderão ser úteis futuramente.

2. DOCUMENTAÇÃO E CONTEXTO — LEITURA OTIMIZADA

2.1. Fonte inicial

A especificação inicial encontra-se no PR #50, especialmente em:

- `workstation/creative-workstation/README.md`
- `workstation/creative-workstation/ARCHITECTURE.md`
- `workstation/creative-workstation/IMPLEMENTATION_PLAN.md`
- `workstation/creative-workstation/INTEGRATIONS.md`
- `workstation/creative-workstation/AGENTS.md`
- `workstation/creative-workstation/IMPLEMENTER_PROMPT.md`

O plano adicional introduz os documentos operacionais:

- `workstation/creative-workstation/EXECUTION_BRIEF.md`
- `workstation/creative-workstation/VERIFICATION_MATRIX.md`
- `workstation/creative-workstation/PHASE_PROMPTS.md`

Confirme a presença e o conteúdo desses arquivos na branch correta.

Se o PR #50 ainda não estiver integrado, consulte-o diretamente pela branch de documentação. **Não faça merge apenas para acessar a especificação.**

2.2. Leitura canônica obrigatória

Antes de modificar qualquer código, leia, na ordem aplicável:

1. `AGENTS.md`
2. `workstation/AGENTS.md`
3. `workstation/context/README.md`
4. Todos os documentos e gates adicionais exigidos por esses arquivos.

Consulte também os documentos canônicos pertinentes:

- `workstation/ROADMAP.md`
- `workstation/SOURCE_MATRIX.md`
- `workstation/ARCHITECTURE.md`
- `workstation/context/CURRENT_STATE.md`
- `workstation/context/DECISIONS.md`
- `workstation/context/HERMES_WORKSTATION_INTELLIGENCE.md`
- `workstation/context/EXPERIENCE_COMPILER.md`

- `workstation/context/engineering-journal/CURRENT.md`

Verifique os caminhos reais e as instruções de precedência dos documentos antes de usá-los.

2.3. Estratégia para economizar contexto e processamento

A leitura deve ser seletiva, mas nunca omitir uma obrigação canônica.

Depois das leituras obrigatórias:

1. Consulte `EXECUTION_BRIEF.md` para recuperar o contexto executivo consolidado.
2. Consulte somente a seção da fase atual em `IMPLEMENTATION_PLAN.md`.
3. Consulte os critérios correspondentes em `VERIFICATION_MATRIX.md`.
4. Consulte o prompt específico dessa fase em `PHASE_PROMPTS.md`.
5. Investigue os owners e os símbolos de código afetados.
6. Carregue documentação adicional somente quando necessária.

Não examine milhares de arquivos sem justificativa. Use buscas por símbolos, caminhos, dependências e contratos.

Não reconstrua decisões já registradas sem evidência de incompatibilidade.

2.4. Precedência em caso de conflito

Respeite primeiro as instruções e políticas canônicas aplicáveis, os gates de segurança e os contratos executáveis vigentes.

Use a documentação da iniciativa como especificação funcional.

Considere o código como evidência do comportamento atual, não como justificativa automática para descartar requisitos.

Se houver conflito material:

- Identifique as fontes conflitantes.
- Registre o requisito e o comportamento real.
- Avalie a menor solução compatível.
- Resolva dentro da autoridade disponível.
- Registre uma decisão arquitetural explícita quando necessária.
- Não silencie requisitos importantes por conveniência.

3. CW-01 — AUDITORIA, BASELINE E LIBERAÇÃO

Objetivo: determinar precisamente o estado do Hermes, localizar os pontos corretos de integração e estabelecer um baseline qualificado.

Esta é a primeira etapa de execução.

3.1. Auditoria do repositório

Identifique:

- HEAD atual do `main`.
- Estado e base do PR #50.
- HEAD da branch de documentação.
- Upstream remoto e SHA fixado.
- Merge-base relevante.
- Alterações pendentes.
- Branches de trabalho relacionadas.
- Checks obrigatórios e situação do CI.
- Políticas vigentes de branch e merge.

Execute o procedimento **H-079 upstream-first** conforme sua definição canônica.

Verifique H-080A, H-080B, KI-024 e os demais bloqueios críticos vigentes.

Não pressuponha que um bloqueio histórico continua aberto. Confirme seu estado atual com evidências.

Não faça implementação funcional sobre um baseline considerado bloqueado pela política canônica.

3.2. Auditoria dos subsistemas

Identifique os owners, seams, símbolos e contratos atuais dos seguintes subsistemas:

Subsistema	Responsabilidade investigada
Control Plane	Autoridade, políticas, verificações
Capability Router	Seleção e encaminhamento de capacidades
TaskCompiler	Compilação e execução operacional
OperationalCapabilityRegistry	Descoberta e reutilização de capacidades
Experience Compiler	Extração, certificação e promoção
ArtifactStore	Persistência e referência de artefatos
ExecutionJournal	Evidências e histórico de execução

Subsistema	Responsabilidade investigada
Session / TaskRun / Kanban	Estado e ciclo de tarefas
BrowserTask / Chromium Electron	Visualização e automação
Plugins / Skills / MCP	Integrações externas
Lifecycle / Health	Processos, recursos e dependências

Investigue inicialmente, quando existentes e pertinentes:

- workstation/capabilities.py
- workstation/control_plane/
- workstation/operational_capabilities.py
- workstation/host.py
- workstation/health.py
- apps/desktop/electron/
- workstation/components.lock.json

Esses caminhos são referências de investigação, não autorização para modificá-los indiscriminadamente.

3.3. Matriz de reutilização

Produza a matriz:

owner → arquivo::símbolo → contrato/comportamento atual → capacidade reutilizável → lacuna concreta → menor alteração necessária

Cubra obrigatoriamente:

- Descoberta e lifecycle de engines.
- MCP e skills.
- Chromium e Electron.
- Execução controlada.
- Persistência e revisões.
- Verificação de artefatos.
- Experience Compiler.
- Permissões, cancelamento e rollback.

Não crie uma nova infraestrutura quando a existente já atender ao contrato.

3.4. Auditoria das tecnologias externas

Verifique fontes oficiais, commits/versões, dependências, riscos e condições de licenciamento para:

- Penpot MCP.
- Penpot AI Kit.
- Three.js Editor.
- FFmpeg e ffprobe.
- Remotion.

Registre compatibilidade com as plataformas-alvo, principalmente Windows e Hermes Desktop/Electron.

Audite também os limites de isolamento, credenciais, filesystem e execução arbitrária.

Não instale as tecnologias durante CW-01.

3.5. Entregas e conclusão da CW-01

Registre os resultados no jornal de engenharia e nos documentos pertinentes.

Forneça:

- SHAs reais.
- Gates e CI verificados.
- Owners e contratos.
- Bloqueios comprovados.
- Falsificadores das principais hipóteses.
- Riscos de integração.
- Lista mínima de arquivos e símbolos a modificar na CW-02.
- Classificação de liberação: `GO` ou `BLOCKED`.

Se `GO`, **inicie CW-02 imediatamente**.

Se `BLOCKED`, resolva os bloqueios reparáveis dentro da autoridade disponível. Se permanecer bloqueado, não altere runtime dependente.

4. CW-02 — CREATIVE RUNTIME

Objetivo: disponibilizar engines criativas ao Hermes pelo mecanismo operacional existente.

Implemente a menor extensão suficiente para:

1. Descobrir ferramentas instaladas.
2. Detectar versões e dependências.
3. Classificar compatibilidade.
4. Solicitar autorização para instalações opcionais.
5. Inicializar e encerrar processos controlados.
6. Executar health checks.
7. Gerenciar portas locais e conflitos.
8. Isolar processos e recursos.
9. Persistir os projetos e referências necessários.
10. Recuperar estado depois de reinicialização.
11. Reportar erros estruturados.
12. Desativar ou remover integrações de forma segura.

Regras de arquitetura

Não crie outro lifecycle manager se o Hermes já tiver um equivalente.

Não crie um registry paralelo ao `OperationalCapabilityRegistry`.

Não crie um novo sistema de sessão ou um novo modelo de autorização.

Utilize Control Plane, Policy, Verifier, ArtifactStore e ExecutionJournal existentes.

Prefira operações tipadas e contratos verificáveis.

Critérios de aceitação

Demonstre descoberta de dependência presente e ausente, versão incompatível, health positivo e negativo, cancelamento, término de processo, porta ocupada, recuperação e impossibilidade de executar ação não autorizada.

Certifique-se de que engines opcionais não sejam instaladas ou iniciadas silenciosamente.

Após testes e qualificações aplicáveis, documente os resultados, abra o PR específico e avance para CW-03A.

5. CW-03A — PRIMEIRA VERTICAL CRIATIVA

Objetivo: provar uma experiência criativa completa antes de adicionar editores complexos.

Fluxo obrigatório

Solicitação → execução React/SVG → preview Chromium → renderização PNG → verificação → persistência → reabertura

O agente deverá:

1. Receber uma solicitação de criação visual.
2. Criar um projeto editável React/SVG.
3. Registrar os fontes e assets.
4. Abrir uma prévia no Chromium real do Hermes Desktop.
5. Gerar um PNG correspondente ao projeto.
6. Verificar dimensões, formato, integridade e hashes.
7. Registrar evidências no ArtifactStore/ExecutionJournal.
8. Preservar o projeto editável.
9. Reabrir o mesmo projeto após reinicialização.

Requisitos adicionais

Separe fontes editáveis dos outputs gerados.

Preserve referências estáveis entre projeto, task, evidências e artefatos.

Não dependa exclusivamente de screenshots ou coordenadas para identificar objetos.

Não substitua um projeto real por um mock que apenas simule o resultado final.

Teste obrigatório

Executar o fluxo completo com o **Chromium real do Hermes Desktop/Electron**, incluindo exportação e reabertura.

Testes unitários não substituem esse E2E.

Se o ambiente atual não permitir Electron real, execute as verificações disponíveis, marque o E2E `NOT_RUN` e não declare `QUALIFIED`.

Após qualificação, avance para CW-03B.

6. CW-03B — FFMPEG E FFPROBE

Objetivo: incorporar processamento audiovisual verificável.

Integre FFmpeg/ffprobe usando operações controladas e contratos tipados.

Capacidades iniciais:

- Inspecionar arquivos de mídia.
- Ler metadados técnicos.
- Identificar codecs, resolução, duração e streams.
- Converter formatos suportados.
- Produzir vídeo a partir de frames.
- Processar áudio quando solicitado.
- Validar arquivos gerados.
- Tratar erros, cancelamentos e processos interrompidos.

Segurança

Não exponha comandos arbitrários sem validação.

Valide caminhos, parâmetros, arquivos de entrada, destinos e escopo de escrita.

Previna writes fora do workspace autorizado.

Evite sobrescritas destrutivas e execuções duplicadas.

Qualificação

Teste arquivos válidos e inválidos, parâmetros incompatíveis, cancelamento, recuperação, outputs e inspeção com ffprobe.

Verifique integridade, dimensões, streams, duração quando aplicável e hashes.

Depois da qualificação, prossiga para CW-03C.

7. CW-03C — REMOTION

Objetivo: permitir motion graphics programáticos e renderização de vídeos editáveis.

Esta integração é **condicionada ao licenciamento e à compatibilidade operacional**.

Primeiro: licenciamento

Verifique:

- Licença oficial vigente.
- Direitos para uso comercial.
- Condições de redistribuição.

- Impacto do modelo de distribuição do Hermes.
- Dependências e custos aplicáveis.
- Permissões e restrições relevantes.

Não presuma que Remotion é MIT ou que sua licença permite qualquer integração.

Se a licença não permitir a integração proposta, registre `BLOCKED` para CW-03C e não incorpore a tecnologia indevidamente.

Esse impedimento não deve bloquear automaticamente Penpot, Three.js ou as outras engines independentes.

Implementação autorizada

Se os requisitos forem atendidos:

1. Criar projetos editáveis de animação React/Remotion.
2. Parametrizar cenas e composições.
3. Visualizar animações no Chromium.
4. Renderizar frames e vídeos.
5. Verificar outputs com os mecanismos existentes.
6. Preservar código, assets, composição e dependências.
7. Permitir alterações posteriores.
8. Registrar evidências e metadados da renderização.

Testes

Demonstre renderização real, reabertura, alteração de parâmetros, cancelamento, erros e verificação do vídeo.

Registre limitações técnicas ou de licença.

Após a qualificação possível dessa etapa, avance para CW-04.

8. CW-04 — PENPOT

Objetivo: integrar uma superfície de design visual estruturado, editável tanto pelo usuário quanto pelo agente.

Fontes iniciais

MCP oficial:

<https://github.com/penpot/penpot/tree/develop/mcp>

AI Kit:

<https://github.com/penpot/penpot-ai-kit>

Confirme contratos, versões e requisitos atuais diretamente nas fontes oficiais.

Capacidades

Implemente, conforme suporte real das interfaces disponíveis:

- Conexão MCP autorizada.
- Descoberta de projetos e arquivos.
- Leitura de layouts.
- Inspeção de componentes.
- Modificação de elementos.
- Manipulação de tokens e estilos.
- Exportação de assets.
- Abertura no Chromium integrado.
- Identificação de versões e revisões.
- Detecção de conflitos de edição.
- Persistência de referências do projeto.

Não presuma que o MCP oferece todas as operações desejadas. Se uma capacidade não existir, identifique a lacuna e implemente somente uma extensão segura e justificada.

Teste obrigatório de colaboração humano/agente

Execute este cenário:

1. O agente cria um layout.
2. O projeto abre no Penpot.
3. O usuário modifica manualmente um elemento.
4. O agente recebe uma nova instrução.
5. O agente detecta o estado atualizado.
6. O agente modifica outra parte do projeto.
7. A alteração humana permanece preservada.

Quando ocorrer conflito de revisão, não sobrescreva silenciosamente as alterações do usuário.

Restrições

O Penpot não deve se tornar uma autoridade paralela para tarefas e políticas do Hermes.

Não instale nem inicialize MCPs automaticamente sem a autorização exigida.

Respeite os limites de credenciais e rede.

Qualifique o fluxo real e avance para CW-05.

9. CW-05 — THREE.JS EDITOR

Objetivo: transformar o editor Three.js em uma superfície 3D editável, controlável por operações tipadas do Hermes.

Fonte oficial:

<https://github.com/mrdoob/three.js>

Estratégia arquitetural

Priorize uma bridge fina entre o Hermes e o editor oficial.

Evite fazer fork quando uma extensão ou adaptador resolver o problema.

Caso seja necessário modificar o editor, documente a incompatibilidade, a justificativa e os testes de compatibilidade upstream.

Contratos desejados

Implemente os métodos aplicáveis:

- `inspectScene`
- `createObject`
- `modifyGeometry`
- `setMaterial`
- `configureLights`
- `configureCamera`
- `addAnimation`
- `importGLB`
- `exportScene`
- `capturePreview`
- `undoTransaction`

Use IDs semânticos estáveis para objetos e controle de revisões.

Experiência esperada

O agente deverá conseguir:

1. Criar uma cena 3D.
2. Adicionar objetos.
3. Modificar geometria e materiais.
4. Configurar luz e câmera.
5. Adicionar animação.
6. Abrir a cena no editor real.
7. Preservar alterações feitas pelo usuário.
8. Salvar o projeto.
9. Reabrir a cena posteriormente.
10. Exportar formatos compatíveis.
11. Capturar e verificar uma prévia.

Segurança

Não exponha execução irrestrita de JavaScript em contexto Electron privilegiado.

Valide mensagens entre contextos, parâmetros e escopos.

Controle mutações de cena, erros parciais e transações.

Não execute retries cegos quando o resultado de uma mutação for incerto.

Teste obrigatório

Criação, edição, preview, exportação, reabertura e recuperação de cena real.

Inclua testes de conflito, objeto inexistente, operação inválida e undo.

Após qualificação, avance para CW-06.

10. CW-06 — ENGINES ESPECIALIZADAS E MANIFESTO DE PROJETOS

Objetivo: consolidar o ecossistema criativo e estabelecer a interoperabilidade possível sem introduzir complexidade desnecessária.

10.1. Engines especializadas

Avalie progressivamente:

- Inkscape CLI, para processamento SVG.
- Blender Python/headless, para modelagem e renderização.
- GIMP/Krita, apenas mediante necessidade concreta.
- FFmpeg/ffprobe, como infraestrutura audiovisual já integrada anteriormente.

Cada engine efetivamente incorporada deverá possuir:

- Contratos tipados.
- Discovery e health.
- Versão e dependências.
- Escopo de filesystem.
- Autorizações.
- Isolamento.
- Cancelamento.
- Verificação dos outputs.
- Recuperação de falhas.
- Rollback e desativação.

Não instale todas as ferramentas antecipadamente.

Se uma engine opcional não for necessária, registre a decisão de adiamento em vez de inventar uma implementação sem caso de uso.

10.2. Tecnologias que permanecem opcionais

Graphite, Tone.js, Strudel, Godot, PartMode, replicad e tecnologias semelhantes permanecem no backlog até existir justificativa técnica e funcional.

Não amplie o escopo apenas porque uma ferramenta parece interessante.

10.3. Creative Project Manifest

Defina e integre um formato de referência versionado que preserve:

- Identificador do projeto.
- Referência do workspace.
- Engine principal e engines auxiliares.
- Fontes editáveis nativas.
- Assets e dependências.
- Parâmetros de criação e render.

- Versões e requisitos.
- Revisões e hashes.
- Outputs produzidos.
- Histórico ou referências de transformações.
- Evidências de execução.
- Referências aos registros existentes do Hermes.

O manifesto deverá suportar projetos com diferentes engines sem exigir uma representação universal destrutiva.

Ele não substitui `TaskRun`, `ArtifactStore`, `ExecutionJournal` nem o registry operacional.

10.4. Interoperabilidade

Quando tecnicamente suportado, permita importação, exportação e transformações controladas entre formatos.

Não prometa conversão perfeita entre:

- Penpot.
- React.
- SVG.
- Three.js.
- Blender.
- Formatos de vídeo e imagem.

Quando uma transformação perder editabilidade, informação ou semântica, registre explicitamente a limitação.

10.5. Qualificação

Teste persistência e recuperação de projetos, resolução de dependências, versões incompatíveis, referência quebrada, outputs e operações suportadas.

Ao concluir, avance para CW-07.

11. CW-07 — EXPERIENCE COMPILER E REUTILIZAÇÃO OPERACIONAL

Objetivo: permitir que o Hermes aprenda a reutilizar procedimentos criativos efetivamente comprovados.

A integração deve usar o **Experience Compiler já existente**, sem criar outro.

Pré-condição

Pelo menos uma vertical criativa precisa estar realmente funcional e possuir evidências verificáveis.

Não transforme uma sequência hipotética em capability certificada.

Pipeline obrigatório

1. **Observação:** registrar operações executadas durante tarefas criativas.
2. **Efeitos:** identificar resultados observáveis e verificáveis.
3. **Mineração:** reconhecer sequências candidatas à reutilização.
4. **Parametrização:** distinguir dados variáveis de operações estáveis.
5. **Candidatura:** gerar candidatos operacionais compatíveis com os contratos existentes.
6. **Validação positiva:** demonstrar resultados esperados.
7. **Validação negativa:** executar controles negativos seguros.
8. **Replay:** reproduzir operações em contexto controlado.
9. **Certificação:** utilizar a política de promoção existente.
10. **Reutilização:** disponibilizar capabilities aprovadas para execuções futuras.

Exemplos de candidatos

- Criar um SVG a partir de parâmetros.
- Exportar layouts com dimensões específicas.
- Aplicar uma configuração parametrizada de iluminação.
- Produzir previews 3D.
- Renderizar uma composição audiovisual.
- Gerar variantes de design sem reconstruir o projeto.

Esses são exemplos, não autorização para certificar operações sem evidência.

Regras de promoção

Uma execução bem-sucedida isolada não basta para promoção automática.

A certificação exige os oráculos e políticas aplicáveis, com replay positivo e controles negativos seguros.

Skills em Markdown podem orientar a execução, mas não substituem capacidades executáveis certificadas.

APIs, MCPs, CLIs e bridges realizam operações; a camada operacional do Hermes determina autoridade e reutilização.

Invalidação

Uma capability deverá ser revalidada quando mudar algum requisito relevante:

- Versão da engine.
- Schema ou contrato.
- Escopo de autorização.
- Dependência necessária.
- Estrutura do projeto.
- Recurso externo.
- Comportamento do verificador.

Não reutilize cegamente uma capability certificada sob condições incompatíveis.

Eficiência

A meta é reduzir chamadas desnecessárias a LLMs, preservando confiabilidade, qualidade e autoridade.

Não substitua o planejamento inteligente por replay indiscriminado.

Demonstre que um procedimento criativo aprovado pode ser reutilizado em uma tarefa posterior, com validação de seu resultado e registro da execução.

12. INVARIANTES ARQUITETURAIS

Estas restrições se aplicam a **todas as fases**.

Componentes que não podem ser duplicados

Não introduza sistemas paralelos de:

- Session.
- TaskRun.
- Kanban.
- BrowserTask.
- Control Plane.
- Approvals e Policy.

- Verifier.
- ExecutionJournal.
- ArtifactStore.
- OperationalCapabilityRegistry.
- Experience Compiler.

Exceções exigem demonstração documentada de insuficiência do contrato atual e uma decisão arquitetural explícita.

Ordem de preferência para execução

Use, nesta ordem, conforme adequação:

1. APIs e operações tipadas.
2. Código, filesystem e CLI controlados.
3. MCPs autorizados.
4. Browser automation, quando interfaces estruturadas não atenderem à necessidade.

O Chromium deve funcionar prioritariamente como superfície de interação e visualização humana, e não como substituto indiscriminado de APIs.

Semântica operacional

Preserve a separação entre:

- Instrução.
- Seleção de capacidade.
- Autorização.
- Execução.
- Verificação.
- Persistência.
- Certificação.
- Reutilização.

Não misture essas responsabilidades para simplificar a implementação.

13. SEGURANÇA, DEPENDÊNCIAS E LICENCIAMENTO

Antes de incorporar cada integração, registre:

- Repositório oficial.

- Commit ou versão fixada.
- Licença e obrigações.
- Condições de uso comercial e redistribuição.
- Dependências diretas e transitivas relevantes.
- Permissões.
- Superfícies de execução.
- Acesso a filesystem, rede e credenciais.
- Possibilidades de execução arbitrária.
- Estratégia de isolamento.
- Estratégia de desativação e rollback.
- Evidências de validação.

Atenção especial à licença própria do Remotion, às condições GPL/AGPL e às licenças independentes de assets ou marcas.

MCPs comunitários e skills externas devem ser auditados.

Não forneça acesso irrestrito à máquina, às credenciais ou ao workspace a componentes externos sem justificativa e autorização.

Proibições operacionais

- Instalações silenciosas.
- Execução privilegiada arbitrária.
- Escrita fora do escopo autorizado.
- Exposição de segredos em logs.
- Sobrescrita silenciosa de alterações humanas.
- Retry cego de operações mutáveis.
- Operações destrutivas sem autorização exigida.
- Atribuição fictícia de resultados a ferramentas.
- Promoção de capabilities sem validação empírica.

14. VERIFICAÇÃO E QUALIFICAÇÃO

`VERIFICATION_MATRIX.md` é a referência operacional dos oráculos, testes negativos, E2E, rollback e gates da iniciativa, subordinada às políticas canônicas aplicáveis.

Execute para cada fase, conforme o subsistema alterado:

- Testes unitários.
- Testes de integração.
- Testes negativos de autorização.
- Testes de isolamento.
- Testes de cancelamento.
- Testes de recuperação.
- Testes de idempotência e operações duplicadas.
- Testes de persistência.
- Testes de revisões e conflitos.
- Verificação de outputs.
- Regressões dos owners afetados.
- Electron E2E quando houver interface.
- Auditoria de first-party seams.
- CI no HEAD exato.
- Verificação final do upstream e baseline.

Não invente comandos. Descubra os comandos reais nas instruções do projeto e nos respectivos workflows.

Não invente `PASS`, CI verde, logs, hashes, screenshots, receipts ou aprovação de licença.

Use `NOT_RUN` quando um teste não foi executado.

Use `BLOCKED` quando o gate impede a continuidade.

Use `PARTIAL` quando uma entrega estiver incompleta.

Distinção entre implementação e qualificação

`IMPLEMENTED` não significa `QUALIFIED`.

Uma fase pode ter código implementado e ainda depender de E2E, testes ou CI.

Não declare conclusão integral quando existirem lacunas obrigatórias sem evidência.

Cada fase deve ter critérios explícitos de entrada, saída e aceite.

15. DISCIPLINA DE GIT E EXECUÇÃO CONTÍNUA

Branches e PRs

Trabalhe em branches específicas por fase ou subfase.

Não reúna toda a iniciativa em um único PR.

Não misture, sem necessidade, sincronização upstream, runtime, integração de editores e Experience Compiler.

Mantenha commits pequenos e descritivos.

Abra PRs revisáveis com descrição de escopo, evidências e dependências.

Continuidade sem merge automático

Como o `main` não pode ser modificado automaticamente, preserve a continuidade usando o mecanismo permitido pelo workflow canônico.

Quando uma fase depender de alterações de um PR ainda não integrado, avalie branches e PRs dependentes (*stacked PRs*), preservando bases e SHAs identificáveis.

Use essa estratégia somente quando compatível com as políticas do repositório.

Não finja que o `main` já contém alterações pendentes.

Não qualifique etapas sobre uma base não verificada.

Se o processo exigir aprovação humana ou merge para permitir com segurança a fase seguinte, registre essa dependência e execute apenas as atividades independentes permitidas.

A ausência de autorização para merge não é autorização para ignorar os gates nem motivo para encerrar prematuramente trabalhos independentes.

Documentação

Depois de cada alteração efetiva, atualize os documentos pertinentes:

- Roadmap.
- Current State.
- Decisions, quando houver decisão nova.
- Engineering Journal.
- Hermes Workstation Intelligence.
- Source Matrix, quando houver mudança de fontes, contratos ou owners.
- Documentação da iniciativa.
- Matrizes de verificação e integração, quando aplicável.

Não marque entregas como concluídas antecipadamente.

Evite duplicação documental. Preserve o papel canônico de cada arquivo.

16. RELATÓRIO OBRIGATÓRIO DE CADA FASE

Ao finalizar cada fase, produza o seguinte registro:

FASE / STATUS

CW-xx — GO | BLOCKED | PARTIAL | IMPLEMENTED | E2E_PASS | QUALIFIED , conforme aplicável.

BASELINE

HEAD, upstream pinado, merge-base, gates e situação do CI no commit exato.

OWNERS

arquivo::símbolo → comportamento reutilizado → alteração realizada .

IMPLEMENTAÇÃO

Arquivos modificados, contratos novos, componentes reutilizados e comportamento entregue.

EXTERNAL

Engines e integrações, versões, licença, riscos e situação real.

TESTS

Comandos efetivamente executados, resultados, negativos e NOT_RUN .

EVIDENCE

Receipts, hashes, revisões, previews, outputs, logs e referências verificáveis.

BLOCKERS

Motivo, impacto, menor correção segura e rollback.

GITHUB

Branch, commits, PR, checks, HEAD e situação de merge.

NEXT

Próxima fase elegível e suas dependências.

Regra de continuidade

Depois de gerar esse registro:

1. Determine se o gate de saída da fase foi atendido.
2. Se foi atendido, inicie automaticamente a fase seguinte.

3. Se não foi atendido, tente resolver as falhas permitidas.
4. Se persistir bloqueio, identifique os trabalhos independentes autorizados.
5. Não represente como concluída uma fase dependente de uma etapa não qualificada.
6. Não peça novas instruções apenas para continuar o plano já autorizado.

Os relatórios intermediários são checkpoints, não encerramentos da missão.

17. CENÁRIOS FINAIS DE ACEITAÇÃO

A implementação consolidada deverá buscar demonstrar os seguintes cenários integrados.

Cenário A — Design editável

O usuário solicita um material visual.

O Hermes produz o projeto, abre o editor compatível no Chromium, permite alterações humanas, preserva revisões e exporta um arquivo verificado.

Cenário B — Animação e vídeo

O usuário solicita uma animação.

O Hermes utiliza capacidades de composição e renderização autorizadas, gera o vídeo, verifica suas propriedades e preserva o projeto original.

Cenário C — Criação 3D

O usuário solicita:

"Crie uma logo 3D, anime sua entrada, adicione iluminação, produza um vídeo de oito segundos e preserve o projeto editável."

O Hermes deverá usar as engines compatíveis e efetivamente disponíveis para executar as operações, abrir a cena ou projeto no Chromium, preservar seus fontes e produzir outputs verificáveis.

Se a cadeia completa depender de uma integração não qualificada, reporte a limitação em vez de simular sucesso.

Cenário D — Reutilização operacional

Depois de executar e certificar adequadamente um procedimento criativo, uma nova solicitação compatível deverá poder reutilizar essa capability pelo mecanismo operacional existente.

Demonstre:

- Seleção da capability.
 - Verificação de suas condições de validade.
 - Execução controlada.
 - Resultado verificável.
 - Evidências persistidas.
 - Comportamento seguro diante de incompatibilidade.
-

18. RELATÓRIO FINAL DA INICIATIVA

Depois de executar todas as fases elegíveis, produza uma consolidação contendo:

1. Status individual de CW-01 a CW-07, incluindo CW-03A/B/C.
2. Matriz de funcionalidades planejadas, implementadas e qualificadas.
3. Componentes existentes reaproveitados.
4. Contratos e componentes novos.
5. Engines integradas e versões efetivamente testadas.
6. PRs, branches, commits e situação dos merges.
7. CI e regressões do HEAD correspondente.
8. Evidências de E2E real.
9. Limitações técnicas e de licenciamento.
10. Bloqueios externos pendentes.
11. Compatibilidade upstream.
12. Situação real do Creative Project Manifest.
13. Situação real da integração ao Experience Compiler.
14. Demonstração dos cenários de aceitação disponíveis.
15. Próximas ações mínimas para alcançar a qualificação integral.

Não declare a iniciativa **QUALIFIED** se alguma integração obrigatória permanecer sem a qualificação exigida.

Diferencie claramente escopo opcional adiado de requisito obrigatório não atendido.

19. INSTRUÇÃO FINAL — EXECUTAR, NÃO REPLANEJAR

A visão do produto, a ordem das fases e as restrições arquiteturais já estão definidas.

Sua responsabilidade é **transformar essa especificação em implementação real, testada, documentada e inspecionável.**

Não desperdice contexto produzindo uma nova arquitetura especulativa.

Não carregue documentação extensa sem necessidade.

Não instale toda a suíte antecipadamente.

Não crie componentes paralelos.

Não substitua testes reais por mocks.

Não declare resultados inexistentes.

Não faça merge no `main` automaticamente.

Não encerre a execução depois da primeira fase aprovada.

COMECE AGORA POR CW-01.

Execute a auditoria H-079, confirme o baseline, libere os gates e identifique os owners e os menores gaps de integração.

Na sequência, implemente CW-02, CW-03A, CW-03B, CW-03C, CW-04, CW-05, CW-06 e CW-07, respeitando a qualificação e as dependências de cada etapa.

A conclusão esperada não é um novo planejamento. É a máxima implementação efetiva e comprovável de toda a iniciativa, até CW-07, dentro das autorizações e dos gates vigentes.